Laboratorio 08 Django REST Framework

FarmaPoint

Desarrollo de Aplicaciones Empresariales

Integrantes: Dario Maza, Sheila Aguirre y Jose Contreras.

Docente: Yunior Bestard Aroche. Revisión: 17 de octubre de 2026.

Este informe desarrolla los 14 ejercicios del documento GLAB_S08_django.pdf sobre la versión actual del repositorio. Se implementó y comprobó la API de productos, ventas y ubicaciones, preservando las páginas HTML y el administrador.

Código base revisado	8c901e4, publicado el 17 de octubre de 2026
Código complementario publicado	c5144bd377d4
Integrantes	Dario Maza; Sheila Aguirre; Jose Contreras
Entorno	Django 5.2.17; Django REST Framework 3.18.3; SQLite
Pruebas de la versión publicada	25 correctas
Pruebas de la copia local con Semana 7	32 correctas
Consultas al serializar ventas	19 antes; 4 después; respuestas iguales
Consultas al serializar evaluaciones	4 antes; 1 después; respuestas iguales
Evidencias	69 PNG; respuestas HTTP JSON; scripts y colección Postman

Repositorio: https://github.com/SAguirreT/django_project

Alcance y método de comprobación

La revisión original del PDF era una propuesta sin ejecución ni capturas. El repositorio consultado hoy ya tenía una implementación parcial. Se conservó ese trabajo y se completaron los recursos y comprobaciones que el PDF exige, sin presentar los resultados anteriores como pruebas de esta ejecución.

Se usó una base SQLite de demostración separada: una categoría, tres productos, tres clientes, tres ventas, seis detalles, tres zonas, tres farmacias y tres evaluaciones. Todos los registros llevan LAB08. Las pruebas automatizadas usan otra base temporal independiente.

Las imágenes de código son capturas de un visor de los archivos reales, con líneas numeradas. Las imágenes de resultados son capturas de JSON producido por peticiones HTTP efectivas al servidor local. Las salidas de serialización proceden de un script Python con Django inicializado, equivalente al código ejecutable en manage.py shell. No se presentan estas imágenes como una terminal ni como Postman.

Las pantallas navegables de DRF se capturaron directamente en Edge. Se adjunta una colección Postman con variables y encadenamiento de IDs para repetir el CRUD. Si se exige estrictamente una captura dentro de Postman o Thunder Client, debe repetirse allí; la evidencia incluida utiliza otro cliente HTTP.

La configuración es local y la API conserva AllowAny de DRF. Las operaciones de escritura de los recursos CRUD no requieren autenticación. Esta entrega no certifica seguridad de producción ni añade facturación, permisos por rol o escritura anidada de ventas.

La copia pública fue preparada desde main para publicar únicamente el trabajo del Lab 08. Los cambios locales previos de Semana 7 se conservaron en la carpeta original y se probaron por separado. Por eso se informan 25 pruebas en publicación y 32 en el entorno local.

Mapa de lectura

01. Instalar DRF e identificar recursos
02. ModelSerializer y serialización
03. Deserialización y validación
04. Vistas genéricas
05. ModelViewSet y DefaultRouter
06. CRUD y códigos HTTP
07. Solo lectura con relación anidada
08. Búsqueda ordenamiento filtro y acción
09. Recursos de la investigación propia
10. Serializers de la investigación
11. ViewSets y códigos HTTP de investigación
12. Recurso anidado y optimización
13. Filtros y acción en investigación
14. Documentación verificación y publicación

Ejercicio 01 Instalar DRF e identificar recursos

Se revisó main en GitHub al 17 de octubre de 2026, revisión 8c901e4. Esta versión ya incorporaba Django REST Framework, ProductoQuerySet.con_stock(), serializers, vistas genéricas y ViewSets. Se integró en la copia local preservando las modificaciones previas de las semanas 6 y 7. Se verificó Django 5.2.17 y DRF 3.18.3, con rest_framework en INSTALLED_APPS.

Producto se utiliza como recurso simple y Venta como recurso relacionado. Zona, Farmacia y EvaluacionUbicacion cubren la investigación de ubicaciones. La API reutiliza los modelos y no añade tablas. La configuración permite elegir una SQLite separada mediante FARMAPOINT_DB. manage.py check no reportó problemas y makemigrations --check --dry-run no detectó cambios.

Justificación. DRF incorpora acceso JSON sin sustituir la interfaz HTML ni el Admin. El PDF de partida afirmaba que no había un QuerySet personalizado: esa descripción se actualizó porque el repositorio de hoy sí lo contiene.

E01_C01_modelos models.py mostrando Producto, Venta, DetalleVenta, Zona y Farmacia.

farmacia/models.py

FarmaPoint · Lab 08 · E01_C01_modelos
Código real del proyecto · líneas originales

```
 28  class Zona(models.Model):
 29      nombre_zona = models.CharField(max_length=100)
 30      distrito = models.CharField(max_length=100)
 31      densidad_poblacional = models.PositiveIntegerField()
 32      flujo_personas = models.CharField(max_length=20)
 33      accesibilidad = models.CharField(max_length=20)
 34
 35      def __str__(self): return self.nombre_zona
 72  class Producto(models.Model):
 73      objects = ProductoQuerySet.as_manager()
 74      nombre = models.CharField(max_length=120)
 75      descripcion = models.TextField(blank=True)
 76      precio = models.DecimalField(max_digits=10, decimal_places=2)
 77      stock = models.PositiveIntegerField()
 78      categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE, related_name='productos')
 79      class Meta: ordering = ['nombre']
 80      def __str__(self): return self.nombre
109  class Venta(models.Model):
110      objects = VentaQuerySet.as_manager()  # Expone los métodos personalizados mediante objects.
111      estado = models.CharField(max_length=10, choices=[('PENDIENTE', 'Pendiente'), ('PAGADO', 'Pagado')],
default='PENDIENTE')  # Diferencia ventas pendientes y confirmadas.
112      cliente = models.ForeignKey(Cliente, on_delete=models.CASCADE, related_name='ventas')
113      productos = models.ManyToManyField(Producto, through='DetalleVenta', related_name='ventas')
114      fecha = models.DateTimeField()
115      total = models.DecimalField(max_digits=10, decimal_places=2)
116      class Meta: ordering = ['-fecha']
117      def __str__(self): return f'Venta #{self.pk} - {self.cliente}'
120  class DetalleVenta(models.Model):
121      venta = models.ForeignKey(Venta, on_delete=models.CASCADE, related_name='detalles')
122      producto = models.ForeignKey(Producto, on_delete=models.CASCADE, related_name='detalles_venta')
123      cantidad = models.PositiveIntegerField()
124      precio_unitario = models.DecimalField(max_digits=10, decimal_places=2)
125      class Meta:
126          ordering = ['venta', 'pk']
127          constraints = [models.UniqueConstraint(fields=['venta', 'producto'], name='detalle_venta_producto_unico')]
128      @property
129      def subtotal(self): return self.cantidad * self.precio_unitario
130      def __str__(self): return f'{self.venta} - {self.producto}'
```

E01_C02_settings settings.py mostrando rest_framework en INSTALLED_APPS.

config/settings.py

FarmaPoint · Lab 08 · E01_C02_settings
Código real del proyecto · líneas originales

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-farmapoint-local-development'
DEBUG = True
ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'farmacia',
]

```

E01_C03_requirements requirements.txt con djangorestframework.

Dependencias instaladas

FarmaPoint · Lab 08 · E01_C03_requirements
Archivo real y versiones verificadas

```
Django==5.2.17
djangorestframework==3.18.3

Python del proyecto: venv/Scripts/python.exe
Django 5.2.17
Django REST Framework 3.18.3
```

Ejercicio 02 ModelSerializer y serialización

ProductoSerializer define id, nombre, descripcion, precio, stock, categoria y categoria_nombre. El identificador categoria permite escribir la relación; categoria_nombre es de solo lectura y obtiene el nombre mediante source="categoria.nombre".

Se serializó un producto y después los tres productos ficticios con many=True. Los resultados reales están en E02_C02_objeto.json y E02_C03_queryset.json. La colección se consultó con select_related("categoria") para evitar consultas adicionales por cada nombre de categoría.

Justificación. El serializer convierte instancias del ORM en tipos representables por el renderizador JSON. many=True indica una colección; no cambia los registros ni sustituye una consulta.

E02_C01_serializer Código de ProductoSerializer en serializers.py.

farmacia/serializers.py

FarmaPoint · Lab 08 · E02_C01_serializer
Código real del proyecto · líneas originales

```
12  class ProductoSerializer(serializers.ModelSerializer):
13      categoria_nombre = serializers.CharField(source='categoria.nombre', read_only=True)
14
15      class Meta:
16          model = Producto
17          fields = ['id', 'nombre', 'descripcion', 'precio', 'stock', 'categoria', 'categoria_nombre']
18
19      def validate_precio(self, value):
20          if value <= 0:
21              raise serializers.ValidationError('El precio debe ser mayor que cero.')
22          return value
```

E02_C02_objeto Shell mostrando la serialización de un producto.

E02_C02_objeto

FarmaPoint · Lab 08 · E02_C02_objeto
Resultado real ejecutado · Python con Django inicializado


```
{
  "expresion": "ProductoSerializer(producto).data",
  "resultado": {
    "id": 1,
    "nombre": "Analgesico A [LAB08]",
    "descripcion": "Producto ficticio de laboratorio.",
    "precio": "5.50",
    "stock": 10,
    "categoria": 1,
    "categoria_nombre": "Salud [LAB08]"
  }
}
```

E02_C03_queryset Shell mostrando la serialización de varios productos.

E02_C03_queryset

FarmaPoint · Lab 08 · E02_C03_queryset
Resultado real ejecutado · Python con Django inicializado


```
{
  "expresion": "ProductoSerializer(Producto.objects.select_related(\"categoria\"), many=True).data",
  "resultado": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 3, "nombre": "Vitamina C [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "8.90",
"stock": 0, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```

Ejercicio 03 Deserialización y validación

validate_precio rechaza valores iguales o menores que cero. La entrada válida usó precio 2.50, stock 1 y una categoría existente. is_valid() devolvió True; validated_data incluyó un Decimal y la instancia relacionada. save() creó un objeto y se comprobó su existencia dentro de una transacción.

La prueba inválida envió precio 0: is_valid() devolvió False y errors incluyó el campo precio. La creación válida se revirtió explícitamente al terminar la demostración, de modo que no dejó un producto temporal. Los datos que fallan validación nunca se guardaron.

Justificación. La validación del serializer protege las entradas de la API antes de persistirlas. No constituye una restricción universal de la base de datos para escrituras que omitan el serializer.

E03_C01_validacion validate_precio dentro de ProductoSerializer.

farmacia/serializers.py

FarmaPoint · Lab 08 · E03_C01_validacion
Código real del proyecto · líneas originales

```
12  class ProductoSerializer(serializers.ModelSerializer):
13      categoria_nombre = serializers.CharField(source='categoria.nombre', read_only=True)
14
15      class Meta:
16          model = Producto
17          fields = ['id', 'nombre', 'descripcion', 'precio', 'stock', 'categoria', 'categoria_nombre']
18
19      def validate_precio(self, value):
20          if value <= 0:
21              raise serializers.ValidationError('El precio debe ser mayor que cero.')
22          return value
```

E03_C02_valido Shell mostrando is_valid() igual a True y validated_data.

E03_C02_valido

FarmaPoint · Lab 08 · E03_C02_valido
Resultado real ejecutado · Python con Django inicializado


```
{
  "is_valid": true,
  "validated_data": {
    "nombre": "Temporal validacion LAB08",
    "descripcion": "",
    "precio": "2.50",
    "stock": 1,
    "categoria": "Salud [LAB08]"
  },
  "save_id": 4,
  "persistido": true,
  "limpieza": "rollback al finalizar la prueba"
}
```

E03_C03_invalido Shell mostrando is_valid() igual a False y errors.

E03_C03_invalido

FarmaPoint · Lab 08 · E03_C03_invalido
Resultado real ejecutado · Python con Django inicializado


```
{
  "is_valid": false,
  "errors": {
    "precio": [
      "El precio debe ser mayor que cero."
    ]
  }
}
```

Ejercicio 04 Vistas genéricas

Se conservaron ProductoListCreateAPIView y ProductoRetrieveUpdateDestroyAPIView del repositorio. La primera resuelve GET y POST de colección; la segunda resuelve GET, PUT, PATCH y DELETE de un producto. Ambas utilizan ProductoSerializer y cargan la categoría con select_related.

Se comprobaron GET /api/v1/basico/productos/ y GET /api/v1/basico/productos/1/, ambos con HTTP 200. Las rutas HTML existentes permanecieron disponibles.

Justificación. El prefijo basico conserva la etapa de vistas genéricas para demostrar el ejercicio. La API principal usa el router en otra dirección; ninguna URL queda registrada dos veces para las mismas operaciones.

E04_C01_vistas Código de las dos vistas genéricas.

farmacia/api_generic_views.py

FarmaPoint · Lab 08 · E04_C01_vistas
Código real del proyecto · líneas originales

```python
from rest_framework import generics

from .models import Producto
from .serializers import ProductoSerializer


class ProductoListCreateAPIView(generics.ListCreateAPIView):
    queryset = Producto.objects.select_related('categoria').all()
    serializer_class = ProductoSerializer


class ProductoRetrieveUpdateDestroyAPIView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Producto.objects.select_related('categoria').all()
    serializer_class = ProductoSerializer
```

E04_C02_urls Rutas de la API.

farmacia/api_generic_urls.py

FarmaPoint · Lab 08 · E04_C02_urls
Código real del proyecto · líneas originales

```python
from django.urls import path

from .api_generic_views import ProductoListCreateAPIView, ProductoRetrieveUpdateDestroyAPIView

urlpatterns = [
    path('', ProductoListCreateAPIView.as_view(), name='api-producto-list-create'),
    path('<int:pk>/', ProductoRetrieveUpdateDestroyAPIView.as_view(), name='api-producto-detail'),
]
```

E04_C03_lista Navegador mostrando la lista JSON de productos.

E04_C03_lista

FarmaPoint · Lab 08 · E04_C03_lista
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/basico/productos/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 3, "nombre": "Vitamina C [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "8.90",
"stock": 0, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```


E04_C04_detalle Navegador mostrando un producto específico.

E04_C04_detalle

FarmaPoint · Lab 08 · E04_C04_detalle
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/basico/productos/1/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": {
    "id": 1,
    "nombre": "Analgesico A [LAB08]",
    "descripcion": "Producto ficticio de laboratorio.",
    "precio": "5.50",
    "stock": 10,
    "categoria": 1,
    "categoria_nombre": "Salud [LAB08]"
  }
}
```

Ejercicio 05 ModelViewSet y DefaultRouter

La API principal utiliza DefaultRouter y registra categorías, productos, farmacias, ventas, zonas y evaluaciones. Los cuatro primeros recursos proceden de la revisión pública; zonas y evaluaciones se incorporaron para cubrir la investigación descrita en el PDF.

GET /api/v1/ devolvió HTTP 200 con los seis recursos. Las operaciones de productos están en /api/v1/productos/ y /api/v1/productos/<id>/. Se conserva el prefijo versionado del repositorio actual en lugar de cambiarlo al /api/ orientativo del documento.

Justificación. ModelViewSet agrupa las acciones CRUD y el router genera los patrones de lista y detalle. La separación por prefijos permite comparar la etapa genérica con la implementación principal sin conflictos de resolución.

E05_C01_viewset Código de ProductoViewSet.

farmacia/api_views.py

FarmaPoint · Lab 08 · E05_C01_viewset
Código real del proyecto · líneas originales

```
17  class ProductoViewSet(viewsets.ModelViewSet):
18      serializer_class = ProductoSerializer
19      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
20      search_fields = ['nombre', 'descripcion', 'categoria__nombre']
21      ordering_fields = ['nombre', 'precio', 'stock']
22      ordering = ['nombre']
23
24      def get_queryset(self):
25          queryset = Producto.objects.select_related('categoria')
26          disponible = self.request.query_params.get('disponible')
27          if disponible and disponible.lower() in {'true', '1', 'si', 'sí'}:
28              queryset = queryset.con_stock()
29          elif disponible and disponible.lower() in {'false', '0', 'no'}:
30              queryset = queryset.filter(stock=0)
31          return queryset
32
33      @action(detail=False, methods=['get'], url_path='con-stock')
34      def con_stock(self, request):
35          productos = self.filter_queryset(self.get_queryset().con_stock())
36          page = self.paginate_queryset(productos)
37          if page is not None:
38              serializer = self.get_serializer(page, many=True)
39              return self.get_paginated_response(serializer.data)
40          return Response(self.get_serializer(productos, many=True).data)
```

E05_C02_router Registro de DefaultRouter y su inclusión bajo /api/.

farmacia/api_urls.py

FarmaPoint · Lab 08 · E05_C02_router
Código real del proyecto · líneas originales

```python
from rest_framework.routers import DefaultRouter

from .api_views import (CategoriaViewSet, FarmaciaViewSet, ProductoViewSet, VentaReadOnlyViewSet,
                        ZonaViewSet, EvaluacionUbicacionReadOnlyViewSet)

router = DefaultRouter()
router.register('categorias', CategoriaViewSet, basename='api-categoria')
router.register('productos', ProductoViewSet, basename='api-producto')
router.register('farmacias', FarmaciaViewSet, basename='api-farmacia')
router.register('ventas', VentaReadOnlyViewSet, basename='api-venta')

router.register('zonas', ZonaViewSet, basename='api-zona')
router.register('evaluaciones', EvaluacionUbicacionReadOnlyViewSet, basename='api-evaluacion')

urlpatterns = router.urls
```

config/urls.py

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/basico/productos/', include('farmacia.api_generic_urls')),
    path('api/v1/', include('farmacia.api_urls')),
    path('', include('farmacia.urls')),
]
```

E05_C03_api_root Índice real de la API en /api/.

E05_C03_api_root

FarmaPoint · Lab 08 · E05_C03_api_root
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": {
    "categorias": "http://127.0.0.1:8008/api/v1/categorias/",
    "productos": "http://127.0.0.1:8008/api/v1/productos/",
    "farmacias": "http://127.0.0.1:8008/api/v1/farmacias/",
    "ventas": "http://127.0.0.1:8008/api/v1/ventas/",
    "zonas": "http://127.0.0.1:8008/api/v1/zonas/",
    "evaluaciones": "http://127.0.0.1:8008/api/v1/evaluaciones/"
  }
}
```

Ejercicio 06 CRUD y códigos HTTP

Se ejecutaron peticiones HTTP reales a productos mediante urllib contra el servidor local de demostración. Se registró método, URL, cuerpo enviado, código y respuesta. El producto temporal creado por POST fue el mismo que se modificó y eliminó.

Resultados: GET lista 200; GET detalle 200; POST válido 201; POST con precio 0, 400; PATCH stock, 200; DELETE, 204 sin cuerpo; GET posterior al mismo ID, 404. También se comprobó en el ORM que el registro borrado ya no existía.

Justificación. Las capturas corresponden a un visor de resultados HTTP, no a Postman ni Thunder Client. Se adjunta una colección importable de Postman para reproducir el recorrido si el docente exige esa interfaz concreta. La API devuelve datos; los templates generan páginas HTML.

Operación	HTTP real
GET lista / detalle	200
POST válido	201
POST inválido	400
PATCH	200
DELETE	204
GET tras eliminar	404

E06_C01_get_lista GET listado y respuesta.

E06_C01_get_lista

FarmaPoint · Lab 08 · E06_C01_get_lista
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 3, "nombre": "Vitamina C [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "8.90",
"stock": 0, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```

E06_C02_get_detalle GET detalle existente.

E06_C02_get_detalle

FarmaPoint · Lab 08 · E06_C02_get_detalle
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/1/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": {
    "id": 1,
    "nombre": "Analgesico A [LAB08]",
    "descripcion": "Producto ficticio de laboratorio.",
    "precio": "5.50",
    "stock": 10,
    "categoria": 1,
    "categoria_nombre": "Salud [LAB08]"
  }
}
```


E06_C03_post_valido POST válido y registro creado.

E06_C03_post_valido

FarmaPoint · Lab 08 · E06_C03_post_valido
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "POST",
  "url": "http://127.0.0.1:8008/api/v1/productos/",
  "cuerpo_enviado": {
    "nombre": "Temporal validacion LAB08",
    "descripcion": "",
    "precio": "2.50",
    "stock": 1,
    "categoria": 1
  },
  "estado_http": 201,
  "respuesta": {
    "id": 4,
    "nombre": "Temporal validacion LAB08",
    "descripcion": "",
    "precio": "2.50",
    "stock": 1,
    "categoria": 1,
    "categoria_nombre": "Salud [LAB08]"
  }
}
```

E06_C04_post_invalido POST inválido y errores de validación.

E06_C04_post_invalido

FarmaPoint · Lab 08 · E06_C04_post_invalido
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "POST",
  "url": "http://127.0.0.1:8008/api/v1/productos/",
  "cuerpo_enviado": {
    "nombre": "Temporal validacion LAB08",
    "descripcion": "",
    "precio": "0",
    "stock": 1,
    "categoria": 1
  },
  "estado_http": 400,
  "respuesta": {
    "precio": [
      "El precio debe ser mayor que cero."
    ]
  }
}
```


E06_C05_patch PATCH/PUT y respuesta.

E06_C05_patch

FarmaPoint · Lab 08 · E06_C05_patch
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "PATCH",
  "url": "http://127.0.0.1:8008/api/v1/productos/4/",
  "cuerpo_enviado": {
    "stock": 3
  },
  "estado_http": 200,
  "respuesta": {
    "id": 4,
    "nombre": "Temporal validacion LAB08",
    "descripcion": "",
    "precio": "2.50",
    "stock": 3,
    "categoria": 1,
    "categoria_nombre": "Salud [LAB08]"
  }
}
```


E06_C06_delete DELETE y estado obtenido.

E06_C06_delete

FarmaPoint · Lab 08 · E06_C06_delete
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "DELETE",
  "url": "http://127.0.0.1:8008/api/v1/productos/4/",
  "cuerpo_enviado": null,
  "estado_http": 204,
  "respuesta": null
}
```

E06_C07_no_encontrado GET de ID inexistente y respuesta.

E06_C07_no_encontrado

FarmaPoint · Lab 08 · E06_C07_no_encontrado
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/4/",
  "cuerpo_enviado": null,
  "estado_http": 404,
  "respuesta": {
    "detail": "No Producto matches the given query."
  }
}
```

Ejercicio 07 Solo lectura con relación anidada

VentaReadSerializer incluye ClienteSerializer y una colección de DetalleVentaReadSerializer. Cada detalle incorpora ProductoSerializer, cantidad, precio unitario y subtotal. VentaReadOnlyViewSet publica lista y detalle y rechaza POST con HTTP 405.

Se serializaron las mismas tres ventas, con seis detalles y sus productos, antes y después de optimizar. La versión sin precarga realizó 19 consultas y la optimizada 4. Ambas respuestas fueron iguales. Se usa select_related("cliente") y prefetch_related("detalles__producto__categoria").

Justificación. Se incluyó categoría en la precarga porque ProductoSerializer muestra categoria_nombre. La medición evalúa serializer.data completo; no mide solamente list(queryset), que omitiría los accesos anidados y produciría una comparación engañosa.

E07_C01_serializer_anidado Serializers de Venta y DetalleVenta.

farmacia/serializers.py

FarmaPoint · Lab 08 · E07_C01_serializer_anidado
Código real del proyecto · líneas originales

```
25  class ClienteSerializer(serializers.ModelSerializer):
26      class Meta:
27          model = Cliente
28          fields = ['id', 'nombre', 'documento', 'telefono', 'correo']
62  class DetalleVentaReadSerializer(serializers.ModelSerializer):
63      producto = ProductoSerializer(read_only=True)
64
65      class Meta:
66          model = DetalleVenta
67          fields = ['id', 'producto', 'cantidad', 'precio_unitario', 'subtotal']
70  class VentaReadSerializer(serializers.ModelSerializer):
71      cliente = ClienteSerializer(read_only=True)
72      detalles = DetalleVentaReadSerializer(many=True, read_only=True)
73
74      class Meta:
75          model = Venta
76          fields = ['id', 'cliente', 'fecha', 'total', 'detalles']
```

E07_C02_queryset_optimizado get_queryset con select_related y prefetch_related.

farmacia/api_views.py

FarmaPoint · Lab 08 · E07_C02_queryset_optimizado
Código real del proyecto · líneas originales

```
69  class VentaReadOnlyViewSet(viewsets.ReadOnlyModelViewSet):
70      serializer_class = VentaReadSerializer
71      queryset = Venta.objects.select_related('cliente').prefetch_related(
72          'detalles__producto__categoria'
73      )
```

E07_C03_consultas_antes Medición sin optimización.

E07_C03_consultas_antes

FarmaPoint · Lab 08 · E07_C03_consultas_antes
Resultado real ejecutado · Python con Django inicializado


```
{
  "registros": 3,
  "consultas": 19,
  "serializacion_completa": true,
  "respuestas_iguales": true,
  "sql": [
    "SELECT \"farmacia_venta\".\"id\", \"farmacia_venta\".\"estado\", \"farmacia_venta\".\"cliente_id\",
\"farmacia_venta\".\"fecha\", \"farmacia_venta\".\"total\" FROM \"farmacia_venta\" ORDER BY \"farmacia_venta\".\"id\"
ASC"
  ],
  "nota_sql": "Se muestra la primera consulta. Todas están en el JSON adjunto."
}
```


E07_C04_consultas_despues Medición con optimización.

E07_C04_consultas_despues

FarmaPoint · Lab 08 · E07_C04_consultas_despues
Resultado real ejecutado · Python con Django inicializado


```
{
  "registros": 3,
  "consultas": 4,
  "serializacion_completa": true,
  "respuestas_iguales": true,
  "sql": [
    "SELECT \"farmacia_venta\".\"id\", \"farmacia_venta\".\"estado\", \"farmacia_venta\".\"cliente_id\",
\"farmacia_venta\".\"fecha\", \"farmacia_venta\".\"total\", \"farmacia_cliente\".\"id\",
\"farmacia_cliente\".\"nombre\", \"farmacia_cliente\".\"documento\", \"farmacia_cliente\".\"telefono\",
\"farmacia_cliente\".\"correo\" FROM \"farmacia_venta\" INNER JOIN \"farmacia_cliente\" ON
(\"farmacia_venta\".\"cliente_id\" = \"farmacia_cliente\".\"id\") ORDER BY \"farmacia_venta\".\"id\" ASC"
  ],
  "nota_sql": "Se muestra la primera consulta. Todas están en el JSON adjunto."
}
```

E07_C05_respuesta_api Respuesta JSON anidada.

E07_C05_respuesta_api

FarmaPoint · Lab 08 · E07_C05_respuesta_api
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/ventas/1/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": {
    "id": 1,
    "cliente": {
      "id": 1,
      "nombre": "Cliente ficticio 1 [LAB08]",
      "documento": "LAB08-1",
      "telefono": "",
      "correo": ""
    },
    "fecha": "2026-10-17T20:00:35.750726-05:00",
    "total": "17.50",
    "detalles": [
      {"id": 1, "producto": {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de
laboratorio.", "precio": "5.50", "stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}, "cantidad": 1,
"precio_unitario": "5.50", "subtotal": 5.5},
      {"id": 2, "producto": {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de
laboratorio.", "precio": "12.00", "stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}, "cantidad": 1,
"precio_unitario": "12.00", "subtotal": 12.0}
    ]
  }
}
```

Ejercicio 08 Búsqueda ordenamiento filtro y acción

ProductoViewSet usa SearchFilter sobre nombre, descripción y categoría; OrderingFilter permite nombre, precio y stock. El parámetro disponible selecciona productos con o sin existencias. La acción con-stock reutiliza ProductoQuerySet.con_stock() y también respeta los filtros.

La búsqueda Analgesico devolvió dos registros; el orden por precio mostró 5.50, 8.90 y 12.00; búsqueda y orden combinados devolvieron 5.50 y 12.00. La acción con-stock devolvió los dos productos cuyo stock es positivo. Farmacia filtra el campo estado con comparación sin distinguir mayúsculas.

Justificación. Los filtros componen consultas reutilizables. Se conserva la acción con-stock existente y se añade activas para Farmacia en el ejercicio 13, cubriendo ambas variantes del repositorio y del PDF.

E08_C01_filtros Configuración de SearchFilter y OrderingFilter.

farmacia/api_views.py

FarmaPoint · Lab 08 · E08_C01_filtros
Código real del proyecto · líneas originales

```
17  class ProductoViewSet(viewsets.ModelViewSet):
18      serializer_class = ProductoSerializer
19      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
20      search_fields = ['nombre', 'descripcion', 'categoria__nombre']
21      ordering_fields = ['nombre', 'precio', 'stock']
22      ordering = ['nombre']
23
24      def get_queryset(self):
25          queryset = Producto.objects.select_related('categoria')
26          disponible = self.request.query_params.get('disponible')
27          if disponible and disponible.lower() in {'true', '1', 'si', 'sí'}:
28              queryset = queryset.con_stock()
29          elif disponible and disponible.lower() in {'false', '0', 'no'}:
30              queryset = queryset.filter(stock=0)
31          return queryset
32
33      @action(detail=False, methods=['get'], url_path='con-stock')
34      def con_stock(self, request):
35          productos = self.filter_queryset(self.get_queryset().con_stock())
36          page = self.paginate_queryset(productos)
37          if page is not None:
38              serializer = self.get_serializer(page, many=True)
39              return self.get_paginated_response(serializer.data)
40          return Response(self.get_serializer(productos, many=True).data)
```

E08_C02_filtro_estado Filtro por estado en get_queryset.

farmacia/api_views.py

FarmaPoint · Lab 08 · E08_C02_filtro_estado
Código real del proyecto · líneas originales

```
43  class FarmaciaViewSet(viewsets.ModelViewSet):
44      serializer_class = FarmaciaSerializer
45      queryset = Farmacia.objects.select_related('zona').all()
46      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
47      search_fields = ['nombre', 'tipo', 'estado', 'zona__nombre_zona']
48      ordering_fields = ['nombre', 'distancia_metros', 'estado']
49      ordering = ['nombre']
50
51      def get_queryset(self):
52          queryset = super().get_queryset()
53          estado = self.request.query_params.get('estado')
54          if estado:
55              queryset = queryset.filter(estado__iexact=estado)
56          return queryset
57
58
59      @action(detail=False, methods=['get'])
60      def activas(self, request):
61          # Reutiliza el QuerySet y conserva búsqueda, ordenamiento y paginación.
62          queryset = self.filter_queryset(self.get_queryset().activas())
63          page = self.paginate_queryset(queryset)
64          if page is not None:
65              return self.get_paginated_response(self.get_serializer(page, many=True).data)
66          return Response(self.get_serializer(queryset, many=True).data)
```

E08_C03_action Método con @action(detail=False).

farmacia/models.py

FarmaPoint · Lab 08 · E08_C03_action
Código real del proyecto · líneas originales

```
14  class ProductoQuerySet(models.QuerySet):
15      """Disponibilidad de productos reutilizada por la API."""
16
17      def con_stock(self):
18          return self.filter(stock__gt=0)
21  class FarmaciaQuerySet(models.QuerySet):
22      """Estados de farmacia sin distinguir mayúsculas."""
23
24      def activas(self):
25          return self.filter(estado__iexact='Activa')
```

E08_C04_search Respuesta de búsqueda.

E08_C04_search

FarmaPoint · Lab 08 · E08_C04_search
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/?search=Analgesico",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```


E08_C05_ordering Respuesta ordenada.

E08_C05_ordering

FarmaPoint · Lab 08 · E08_C05_ordering
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/?ordering=precio",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 3, "nombre": "Vitamina C [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "8.90",
"stock": 0, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```


E08_C06_combinacion URL con varios parámetros separados por &.

E08_C06_combinacion

FarmaPoint · Lab 08 · E08_C06_combinacion
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/?search=Analgesico&ordering=precio&disponible=true",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```

E08_C07_action_resultado Respuesta real de la acción.

E08_C07_action_resultado

FarmaPoint · Lab 08 · E08_C07_action_resultado
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/productos/con-stock/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Analgesico A [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "5.50",
"stock": 10, "categoria": 1, "categoria_nombre": "Salud [LAB08]"},
    {"id": 2, "nombre": "Analgesico B [LAB08]", "descripcion": "Producto ficticio de laboratorio.", "precio": "12.00",
"stock": 6, "categoria": 1, "categoria_nombre": "Salud [LAB08]"}
  ]
}
```

Ejercicio 09 Recursos de la investigación propia

La investigación de ubicaciones utiliza Zona, Farmacia y EvaluacionUbicacion. Zona describe distrito, población, flujo y accesibilidad. Farmacia registra nombre, tipo, distancia y estado. EvaluacionUbicacion guarda competencia, centros de salud, alquiler, ventas estimadas y viabilidad.

Farmacia y EvaluacionUbicacion tienen ForeignKey a Zona: una zona puede tener muchas farmacias y evaluaciones. La base de demostración contiene tres registros de cada modelo. La base local no tenía farmacias al revisarla; los valores Activa, Inactiva y ACTIVA se introdujeron expresamente como datos ficticios.

Justificación. Se conservan además las siete entidades del módulo comercial. No se duplican modelos para implementar la API. La tabla de equivalencias documenta los recursos, relaciones, validaciones y QuerySets utilizados.

E09_C01_modelos Modelos Zona, Farmacia y EvaluacionUbicacion.

farmacia/models.py

FarmaPoint · Lab 08 · E09_C01_modelos
Código real del proyecto · líneas originales

```
28  class Zona(models.Model):
29      nombre_zona = models.CharField(max_length=100)
30      distrito = models.CharField(max_length=100)
31      densidad_poblacional = models.PositiveIntegerField()
32      flujo_personas = models.CharField(max_length=20)
33      accesibilidad = models.CharField(max_length=20)
34
35      def __str__(self): return self.nombre_zona
38  class Farmacia(models.Model):
39      objects = FarmaciaQuerySet.as_manager()
40      nombre = models.CharField(max_length=100)
41      tipo = models.CharField(max_length=50)
42      zona = models.ForeignKey(Zona, on_delete=models.CASCADE, related_name='farmacias')
43      distancia_metros = models.PositiveIntegerField()
44      estado = models.CharField(max_length=20)
45
46      def __str__(self): return self.nombre
52  class EvaluacionUbicacion(models.Model):
53      zona = models.ForeignKey(Zona, on_delete=models.CASCADE, related_name='evaluaciones')
54      inkafarmas_cercanos = models.PositiveIntegerField()
55      competidores_cercanos = models.PositiveIntegerField()
56      centros_salud_cercanos = models.PositiveIntegerField()
57      costo_alquiler = models.DecimalField(max_digits=10, decimal_places=2)
58      ventas_estimadas = models.DecimalField(max_digits=10, decimal_places=2)
59      nivel_viabilidad = models.CharField(max_length=10, choices=NIVEL_VIABILIDAD_CHOICES)
60      fecha_creacion = models.DateTimeField(auto_now_add=True)
61
62      def __str__(self): return f'Evaluación #{self.pk} - Zona {self.zona.nombre_zona} ({self.nivel_viabilidad})'
```

E09_C02_relaciones ForeignKey entre los modelos.

farmacia/models.py

FarmaPoint · Lab 08 · E09_C02_relaciones
Código real del proyecto · líneas originales

```
38  class Farmacia(models.Model):
39      objects = FarmaciaQuerySet.as_manager()
40      nombre = models.CharField(max_length=100)
41      tipo = models.CharField(max_length=50)
42      zona = models.ForeignKey(Zona, on_delete=models.CASCADE, related_name='farmacias')
43      distancia_metros = models.PositiveIntegerField()
44      estado = models.CharField(max_length=20)
45
46      def __str__(self): return self.nombre
52  class EvaluacionUbicacion(models.Model):
53      zona = models.ForeignKey(Zona, on_delete=models.CASCADE, related_name='evaluaciones')
54      inkafarmas_cercanos = models.PositiveIntegerField()
55      competidores_cercanos = models.PositiveIntegerField()
56      centros_salud_cercanos = models.PositiveIntegerField()
57      costo_alquiler = models.DecimalField(max_digits=10, decimal_places=2)
58      ventas_estimadas = models.DecimalField(max_digits=10, decimal_places=2)
59      nivel_viabilidad = models.CharField(max_length=10, choices=NIVEL_VIABILIDAD_CHOICES)
60      fecha_creacion = models.DateTimeField(auto_now_add=True)
61
62      def __str__(self): return f'Evaluación #{self.pk} - Zona {self.zona.nombre_zona} ({self.nivel_viabilidad})'
```

E09_C03_tabla_equivalencias Tabla de equivalencias completada.

E09_C03_tabla_equivalencias

FarmaPoint · Lab 08 · E09_C03_tabla_equivalencias
Resultado real ejecutado · Python con Django inicializado


```
{
  "simple": "Producto; precio > 0",
  "relacionado": "Venta -> DetalleVenta -> Producto",
  "investigacion": [
    "Zona",
    "Farmacia",
    "EvaluacionUbicacion"
  ],
  "estado": "Farmacia.estado",
  "relaciones": [
    "Zona 1:N Farmacia",
    "Zona 1:N EvaluacionUbicacion"
  ],
  "querysets": [
    "ProductoQuerySet.con_stock()",
    "FarmaciaQuerySet.activas()"
  ],
  "cantidades": {
    "zonas": 3,
    "farmacias": 3,
    "evaluaciones": 3
  },
  "estados_demo": [
    "Activa",
    "Inactiva",
    "ACTIVA"
  ],
  "nota": "Antes de sembrar, no había farmacias locales. Los estados son datos ficticios explícitos."
}
```

Ejercicio 10 Serializers de la investigación

Se implementaron ZonaSerializer, FarmaciaSerializer y EvaluacionUbicacionSerializer. Farmacia recibe la zona por su identificador y muestra zona_nombre de solo lectura. EvaluacionUbicacion presenta una ZonaSerializer anidada y se expone únicamente para lectura.

FarmaciaSerializer valida distancia_metros > 0. La entrada de 100 metros devolvió is_valid=True; la de 0 metros devolvió False y un error del campo. Se mantienen las opciones Alta, Media y Baja del modelo de evaluación y los tipos definidos en el ORM.

Justificación. Cada serializer define la representación de su recurso. La validación del campo numérico se realiza antes de crear o actualizar. Un campo anidado read_only no permite escribir automáticamente el objeto relacionado.

E10_C01_serializers Los tres serializers.

farmacia/serializers.py

FarmaPoint · Lab 08 · E10_C01_serializers
Código real del proyecto · líneas originales

```
31  class FarmaciaSerializer(serializers.ModelSerializer):
32      zona_nombre = serializers.CharField(source='zona.nombre_zona', read_only=True)
33
34      class Meta:
35          model = Farmacia
36          fields = ['id', 'nombre', 'tipo', 'zona', 'zona_nombre', 'distancia_metros', 'estado']
37
38
39      def validate_distancia_metros(self, value):
40          if value <= 0:
41              raise serializers.ValidationError('La distancia debe ser mayor que cero.')
42          return value
45  class ZonaSerializer(serializers.ModelSerializer):
46      class Meta:
47          model = Zona
48          fields = ['id', 'nombre_zona', 'distrito', 'densidad_poblacional', 'flujo_personas', 'accesibilidad']
51  class EvaluacionUbicacionSerializer(serializers.ModelSerializer):
52      # La zona se muestra anidada; este recurso se publica solo para lectura.
53      zona = ZonaSerializer(read_only=True)
54
55      class Meta:
56          model = EvaluacionUbicacion
57          fields = ['id', 'zona', 'inkafarmas_cercanos', 'competidores_cercanos',
58                    'centros_salud_cercanos', 'costo_alquiler', 'ventas_estimadas',
59                    'nivel_viabilidad', 'fecha_creacion']
```

E10_C02_validacion_valida Prueba válida.

E10_C02_validacion_valida

FarmaPoint · Lab 08 · E10_C02_validacion_valida
Resultado real ejecutado · Python con Django inicializado


```
{
  "datos": {
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": 1,
    "distancia_metros": 100,
    "estado": "Activa"
  },
  "is_valid": true,
  "validated_data": {
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": "Zona 1 [LAB08]",
    "distancia_metros": 100,
    "estado": "Activa"
  },
  "errors": {

  }
}
```


E10_C03_validacion_invalida Prueba inválida.

E10_C03_validacion_invalida

FarmaPoint · Lab 08 · E10_C03_validacion_invalida
Resultado real ejecutado · Python con Django inicializado


```
{
  "datos": {
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": 1,
    "distancia_metros": 0,
    "estado": "Activa"
  },
  "is_valid": false,
  "validated_data": {

  },
  "errors": {
    "distancia_metros": [
      "La distancia debe ser mayor que cero."
    ]
  }
}
```

Ejercicio 11 ViewSets y códigos HTTP de investigación

ZonaViewSet y FarmaciaViewSet ofrecen operaciones CRUD mediante ModelViewSet. Sus registros se añadieron al mismo DefaultRouter utilizado por la API principal. Los detalles se obtienen mediante el identificador de cada recurso.

Las peticiones HTTP a farmacias produjeron 200 en GET, 201 al crear, 400 con distancia 0, 204 al eliminar el registro temporal y 404 al consultar ese ID eliminado. El CRUD de zonas y el PATCH de ambos recursos se comprobaron además en pruebas automatizadas que verifican los valores persistidos.

Justificación. Los códigos permiten distinguir una operación válida, un dato rechazado y un recurso inexistente. DELETE se ejecutó solo sobre el registro creado para la prueba. La eliminación de zonas conserva las reglas CASCADE ya definidas por el modelo.

E11_C01_viewsets Código de los ViewSets.

farmacia/api_views.py

FarmaPoint · Lab 08 · E11_C01_viewsets
Código real del proyecto · líneas originales

```
43  class FarmaciaViewSet(viewsets.ModelViewSet):
44      serializer_class = FarmaciaSerializer
45      queryset = Farmacia.objects.select_related('zona').all()
46      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
47      search_fields = ['nombre', 'tipo', 'estado', 'zona__nombre_zona']
48      ordering_fields = ['nombre', 'distancia_metros', 'estado']
49      ordering = ['nombre']
50
51      def get_queryset(self):
52          queryset = super().get_queryset()
53          estado = self.request.query_params.get('estado')
54          if estado:
55              queryset = queryset.filter(estado__iexact=estado)
56          return queryset
57
58
59      @action(detail=False, methods=['get'])
60      def activas(self, request):
61          # Reutiliza el QuerySet y conserva búsqueda, ordenamiento y paginación.
62          queryset = self.filter_queryset(self.get_queryset().activas())
63          page = self.paginate_queryset(queryset)
64          if page is not None:
65              return self.get_paginated_response(self.get_serializer(page, many=True).data)
66          return Response(self.get_serializer(queryset, many=True).data)
76  class ZonaViewSet(viewsets.ModelViewSet):
77      queryset = Zona.objects.order_by('nombre_zona', 'pk')
78      serializer_class = ZonaSerializer
```

E11_C02_router Registro de Zona y Farmacia.

farmacia/api_urls.py

FarmaPoint · Lab 08 · E11_C02_router
Código real del proyecto · líneas originales

```
from rest_framework.routers import DefaultRouter

from .api_views import (CategoriaViewSet, FarmaciaViewSet, ProductoViewSet, VentaReadOnlyViewSet,
                        ZonaViewSet, EvaluacionUbicacionReadOnlyViewSet)

router = DefaultRouter()
router.register('categorias', CategoriaViewSet, basename='api-categoria')
router.register('productos', ProductoViewSet, basename='api-producto')
router.register('farmacias', FarmaciaViewSet, basename='api-farmacia')
router.register('ventas', VentaReadOnlyViewSet, basename='api-venta')

router.register('zonas', ZonaViewSet, basename='api-zona')
router.register('evaluaciones', EvaluacionUbicacionReadOnlyViewSet, basename='api-evaluacion')

urlpatterns = router.urls
```

E11_C03_http_200 GET exitoso.

E11_C03_http_200

FarmaPoint · Lab 08 · E11_C03_http_200
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Botica 1 [LAB08]", "tipo": "Independiente", "zona": 1, "zona_nombre": "Zona 1 [LAB08]",
"distancia_metros": 150, "estado": "Activa"},
    {"id": 2, "nombre": "Botica 2 [LAB08]", "tipo": "Independiente", "zona": 2, "zona_nombre": "Zona 2 [LAB08]",
"distancia_metros": 300, "estado": "Inactiva"},
    {"id": 3, "nombre": "Botica 3 [LAB08]", "tipo": "Independiente", "zona": 3, "zona_nombre": "Zona 3 [LAB08]",
"distancia_metros": 450, "estado": "ACTIVA"}
  ]
}
```

E11_C04_http_201 POST creado.

E11_C04_http_201

FarmaPoint · Lab 08 · E11_C04_http_201
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "POST",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/",
  "cuerpo_enviado": {
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": 1,
    "distancia_metros": 100,
    "estado": "Activa"
  },
  "estado_http": 201,
  "respuesta": {
    "id": 4,
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": 1,
    "zona_nombre": "Zona 1 [LAB08]",
    "distancia_metros": 100,
    "estado": "Activa"
  }
}
```


E11_C05_http_400 POST inválido.

E11_C05_http_400

FarmaPoint · Lab 08 · E11_C05_http_400
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "POST",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/",
  "cuerpo_enviado": {
    "nombre": "Botica temporal [LAB08]",
    "tipo": "Independiente",
    "zona": 1,
    "distancia_metros": 0,
    "estado": "Activa"
  },
  "estado_http": 400,
  "respuesta": {
    "distancia_metros": [
      "La distancia debe ser mayor que cero."
    ]
  }
}
```

E11_C06_http_404 Recurso inexistente.

E11_C06_http_404

FarmaPoint · Lab 08 · E11_C06_http_404
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/4/",
  "cuerpo_enviado": null,
  "estado_http": 404,
  "respuesta": {
    "detail": "No Farmacia matches the given query."
  }
}
```


E11_C07_http_204 DELETE exitoso.

E11_C07_http_204

FarmaPoint · Lab 08 · E11_C07_http_204
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "DELETE",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/4/",
  "cuerpo_enviado": null,
  "estado_http": 204,
  "respuesta": null
}
```

Ejercicio 12 Recurso anidado y optimización

EvaluacionUbicacionReadOnlyViewSet usa EvaluacionUbicacionSerializer con la zona anidada. get_queryset() aplica select_related("zona") y un orden estable por clave primaria. La API responde con la evaluación y la información de su zona en el mismo JSON.

Sobre las tres evaluaciones ficticias, la serialización completa realizó 4 consultas sin optimizar y 1 con select_related. Se comprobó igualdad de respuestas. GET de evaluación devolvió 200; POST devolvió 405. Las pruebas automatizadas también rechazan PATCH y DELETE.

Justificación. La relación es una ForeignKey y puede resolverse con una unión SQL. Esto evita la consulta adicional de zona por cada evaluación y mantiene la salida de solo lectura.

E12_C01_serializer Serializer anidado.

farmacia/serializers.py

FarmaPoint · Lab 08 · E12_C01_serializer
Código real del proyecto · líneas originales

```
45  class ZonaSerializer(serializers.ModelSerializer):
46      class Meta:
47          model = Zona
48          fields = ['id', 'nombre_zona', 'distrito', 'densidad_poblacional', 'flujo_personas', 'accesibilidad']
51  class EvaluacionUbicacionSerializer(serializers.ModelSerializer):
52      # La zona se muestra anidada; este recurso se publica solo para lectura.
53      zona = ZonaSerializer(read_only=True)
54
55      class Meta:
56          model = EvaluacionUbicacion
57          fields = ['id', 'zona', 'inkafarmas_cercanos', 'competidores_cercanos',
58                    'centros_salud_cercanos', 'costo_alquiler', 'ventas_estimadas',
59                    'nivel_viabilidad', 'fecha_creacion']
```

E12_C02_queryset get_queryset optimizado.

farmacia/api_views.py

FarmaPoint · Lab 08 · E12_C02_queryset
Código real del proyecto · líneas originales

```
81  class EvaluacionUbicacionReadOnlyViewSet(viewsets.ReadOnlyModelViewSet):
82      serializer_class = EvaluacionUbicacionSerializer
83
84      def get_queryset(self):
85          # Carga evaluación y zona juntas para evitar una consulta por fila.
86          return EvaluacionUbicacion.objects.select_related('zona').order_by('pk')
```

E12_C03_antes Medición antes.

E12_C03_antes

FarmaPoint · Lab 08 · E12_C03_antes
Resultado real ejecutado · Python con Django inicializado


```
{
  "registros": 3,
  "consultas": 4,
  "serializacion_completa": true,
  "respuestas_iguales": true,
  "sql": [
    "SELECT \"farmacia_evaluacionubicacion\".\"id\", \"farmacia_evaluacionubicacion\".\"zona_id\",
\"farmacia_evaluacionubicacion\".\"inkafarmas_cercanos\", \"farmacia_evaluacionubicacion\".\"competidores_cercanos\",
\"farmacia_evaluacionubicacion\".\"centros_salud_cercanos\", \"farmacia_evaluacionubicacion\".\"costo_alquiler\",
\"farmacia_evaluacionubicacion\".\"ventas_estimadas\", \"farmacia_evaluacionubicacion\".\"nivel_viabilidad\",
\"farmacia_evaluacionubicacion\".\"fecha_creacion\" FROM \"farmacia_evaluacionubicacion\" ORDER BY
\"farmacia_evaluacionubicacion\".\"id\" ASC"
  ],
  "nota_sql": "Se muestra la primera consulta. Todas están en el JSON adjunto."
}
```


E12_C04_despues Medición después.

E12_C04_despues

FarmaPoint · Lab 08 · E12_C04_despues
Resultado real ejecutado · Python con Django inicializado


```
{
  "registros": 3,
  "consultas": 1,
  "serializacion_completa": true,
  "respuestas_iguales": true,
  "sql": [
    "SELECT \"farmacia_evaluacionubicacion\".\"id\", \"farmacia_evaluacionubicacion\".\"zona_id\",
\"farmacia_evaluacionubicacion\".\"inkafarmas_cercanos\", \"farmacia_evaluacionubicacion\".\"competidores_cercanos\",
\"farmacia_evaluacionubicacion\".\"centros_salud_cercanos\", \"farmacia_evaluacionubicacion\".\"costo_alquiler\",
\"farmacia_evaluacionubicacion\".\"ventas_estimadas\", \"farmacia_evaluacionubicacion\".\"nivel_viabilidad\",
\"farmacia_evaluacionubicacion\".\"fecha_creacion\", \"farmacia_zona\".\"id\", \"farmacia_zona\".\"nombre_zona\",
\"farmacia_zona\".\"distrito\", \"farmacia_zona\".\"densidad_poblacional\", \"farmacia_zona\".\"flujo_personas\",
\"farmacia_zona\".\"accesibilidad\" FROM \"farmacia_evaluacionubicacion\" INNER JOIN \"farmacia_zona\" ON
(\"farmacia_evaluacionubicacion\".\"zona_id\" = \"farmacia_zona\".\"id\") ORDER BY
\"farmacia_evaluacionubicacion\".\"id\" ASC"
  ],
  "nota_sql": "Se muestra la primera consulta. Todas están en el JSON adjunto."
}
```

E12_C05_respuesta JSON con datos de Zona.

E12_C05_respuesta

FarmaPoint · Lab 08 · E12_C05_respuesta
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/evaluaciones/1/",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": {
    "id": 1,
    "zona": {
      "id": 1,
      "nombre_zona": "Zona 1 [LAB08]",
      "distrito": "Lima",
      "densidad_poblacional": 1000,
      "flujo_personas": "Alto",
      "accesibilidad": "Buena"
    },
    "inkafarmas_cercanos": 1,
    "competidores_cercanos": 2,
    "centros_salud_cercanos": 2,
    "costo_alquiler": "1500.00",
    "ventas_estimadas": "6500.00",
    "nivel_viabilidad": "Alta",
    "fecha_creacion": "2026-10-17T20:00:35.748717-05:00"
  }
}
```

Ejercicio 13 Filtros y acción en investigación

FarmaciaViewSet permite búsqueda por nombre, tipo, estado y nombre de zona, y orden por nombre, distancia o estado. get_queryset() aplica ?estado= mediante estado__iexact. La acción activas reutiliza FarmaciaQuerySet.activas().

La búsqueda Botica devolvió tres registros, ordenados por distancia en 150, 300 y 450 metros. ?estado=activa devolvió dos. /activas/?search=Botica&ordering=distancia_metros devolvió las activas de 150 y 450 metros. La prueba automatizada verifica también que una condición contradictoria estado=Inactiva en /activas/ devuelve una lista vacía.

Justificación. La acción pasa por filter_queryset y conserva la paginación si se configura más adelante. La regla del QuerySet evita repetir la lógica de selección de farmacias activas.

E13_C01_filtros Configuración de filtros.

farmacia/api_views.py

FarmaPoint · Lab 08 · E13_C01_filtros
Código real del proyecto · líneas originales

```
43  class FarmaciaViewSet(viewsets.ModelViewSet):
44      serializer_class = FarmaciaSerializer
45      queryset = Farmacia.objects.select_related('zona').all()
46      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
47      search_fields = ['nombre', 'tipo', 'estado', 'zona__nombre_zona']
48      ordering_fields = ['nombre', 'distancia_metros', 'estado']
49      ordering = ['nombre']
50
51      def get_queryset(self):
52          queryset = super().get_queryset()
53          estado = self.request.query_params.get('estado')
54          if estado:
55              queryset = queryset.filter(estado__iexact=estado)
56          return queryset
57
58
59      @action(detail=False, methods=['get'])
60      def activas(self, request):
61          # Reutiliza el QuerySet y conserva búsqueda, ordenamiento y paginación.
62          queryset = self.filter_queryset(self.get_queryset().activas())
63          page = self.paginate_queryset(queryset)
64          if page is not None:
65              return self.get_paginated_response(self.get_serializer(page, many=True).data)
66          return Response(self.get_serializer(queryset, many=True).data)
```

E13_C02_queryset Filtro por parámetro.

farmacia/models.py

FarmaPoint · Lab 08 · E13_C02_queryset
Código real del proyecto · líneas originales

```
21  class FarmaciaQuerySet(models.QuerySet):
22      """Estados de farmacia sin distinguir mayúsculas."""
23
24      def activas(self):
25          return self.filter(estado__iexact='Activa')
```

E13_C03_action Acción personalizada.

farmacia/api_views.py

FarmaPoint · Lab 08 · E13_C03_action
Código real del proyecto · líneas originales

```
43  class FarmaciaViewSet(viewsets.ModelViewSet):
44      serializer_class = FarmaciaSerializer
45      queryset = Farmacia.objects.select_related('zona').all()
46      filter_backends = [filters.SearchFilter, filters.OrderingFilter]
47      search_fields = ['nombre', 'tipo', 'estado', 'zona__nombre_zona']
48      ordering_fields = ['nombre', 'distancia_metros', 'estado']
49      ordering = ['nombre']
50
51      def get_queryset(self):
52          queryset = super().get_queryset()
53          estado = self.request.query_params.get('estado')
54          if estado:
55              queryset = queryset.filter(estado__iexact=estado)
56          return queryset
57
58
59      @action(detail=False, methods=['get'])
60      def activas(self, request):
61          # Reutiliza el QuerySet y conserva búsqueda, ordenamiento y paginación.
62          queryset = self.filter_queryset(self.get_queryset().activas())
63          page = self.paginate_queryset(queryset)
64          if page is not None:
65              return self.get_paginated_response(self.get_serializer(page, many=True).data)
66          return Response(self.get_serializer(queryset, many=True).data)
```

E13_C04_search Búsqueda real.

E13_C04_search

FarmaPoint · Lab 08 · E13_C04_search
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/?search=Botica",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Botica 1 [LAB08]", "tipo": "Independiente", "zona": 1, "zona_nombre": "Zona 1 [LAB08]",
"distancia_metros": 150, "estado": "Activa"},
    {"id": 2, "nombre": "Botica 2 [LAB08]", "tipo": "Independiente", "zona": 2, "zona_nombre": "Zona 2 [LAB08]",
"distancia_metros": 300, "estado": "Inactiva"},
    {"id": 3, "nombre": "Botica 3 [LAB08]", "tipo": "Independiente", "zona": 3, "zona_nombre": "Zona 3 [LAB08]",
"distancia_metros": 450, "estado": "ACTIVA"}
  ]
}
```

E13_C05_ordering Ordenamiento real.

E13_C05_ordering

FarmaPoint · Lab 08 · E13_C05_ordering
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/?ordering=distancia_metros",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Botica 1 [LAB08]", "tipo": "Independiente", "zona": 1, "zona_nombre": "Zona 1 [LAB08]",
"distancia_metros": 150, "estado": "Activa"},
    {"id": 2, "nombre": "Botica 2 [LAB08]", "tipo": "Independiente", "zona": 2, "zona_nombre": "Zona 2 [LAB08]",
"distancia_metros": 300, "estado": "Inactiva"},
    {"id": 3, "nombre": "Botica 3 [LAB08]", "tipo": "Independiente", "zona": 3, "zona_nombre": "Zona 3 [LAB08]",
"distancia_metros": 450, "estado": "ACTIVA"}
  ]
}
```


E13_C06_filtro_parametro Filtro estado.

E13_C06_filtro_parametro

FarmaPoint · Lab 08 · E13_C06_filtro_parametro
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/?estado=activa",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Botica 1 [LAB08]", "tipo": "Independiente", "zona": 1, "zona_nombre": "Zona 1 [LAB08]",
"distancia_metros": 150, "estado": "Activa"},
    {"id": 3, "nombre": "Botica 3 [LAB08]", "tipo": "Independiente", "zona": 3, "zona_nombre": "Zona 3 [LAB08]",
"distancia_metros": 450, "estado": "ACTIVA"}
  ]
}
```


E13_C07_action_resultado Resultado de la acción.

E13_C07_action_resultado

FarmaPoint · Lab 08 · E13_C07_action_resultado
Resultado real ejecutado · HTTP local (urllib)


```
{
  "metodo": "GET",
  "url": "http://127.0.0.1:8008/api/v1/farmacias/activas/?search=Botica&ordering=distancia_metros",
  "cuerpo_enviado": null,
  "estado_http": 200,
  "respuesta": [
    {"id": 1, "nombre": "Botica 1 [LAB08]", "tipo": "Independiente", "zona": 1, "zona_nombre": "Zona 1 [LAB08]",
"distancia_metros": 150, "estado": "Activa"},
    {"id": 3, "nombre": "Botica 3 [LAB08]", "tipo": "Independiente", "zona": 3, "zona_nombre": "Zona 3 [LAB08]",
"distancia_metros": 450, "estado": "ACTIVA"}
  ]
}
```

Ejercicio 14 Documentación verificación y publicación

Se actualizaron los README y los documentos de evidencia del Lab 08, las dependencias y las instrucciones de ejecución. El informe contiene los tres integrantes solicitados. Se revisaron los cambios y se publicaron con autorización expresa del usuario en main.

El código se publicó en el commit c5144bd377d45efcff114b49ab1e0249ed589071. La copia de publicación, basada en main, ejecutó 25 pruebas correctas. La copia local conservó los cambios adicionales de Semana 7 y ejecutó 32 pruebas correctas. Ambas comprobaciones de esquema finalizaron sin cambios. Las diferencias de cantidad corresponden a esas pruebas locales adicionales.

Justificación. Las capturas incluyen el estado previo, commit y push reales. La publicación excluye bases SQLite, respaldos, entornos virtuales y archivos temporales. El informe y las evidencias se incorporan en un segundo commit documental para no alterar la referencia del código verificado.

E14_C01_requirements Dependencias finales.

Dependencias instaladas

FarmaPoint · Lab 08 · E14_C01_requirements
Archivo real y versiones verificadas

```
Django==5.2.17
djangorestframework==3.18.3

Python del proyecto: venv/Scripts/python.exe
Django 5.2.17
Django REST Framework 3.18.3
```

E14_C02_readme README actualizado.

README actualizado

FarmaPoint · Lab 08 · E14_C02_readme
Extracto del archivo real

```
## Laboratorio 08 — Django REST Framework

Actualización del 17 de octubre de 2026. Integrantes: Dario Maza, Sheila Aguirre y Jose Contreras.

La aplicación ejecutable está en `farmacia_project/src`. Conserva las páginas HTML y `/admin/` y añade la API
versionada `/api/v1/`.

| Recurso | Ruta | Operaciones |
|---|---|---|
| Categorías | `/api/v1/categorias/` | CRUD |
| Productos | `/api/v1/productos/` | CRUD; precio mayor que cero |
| Zonas | `/api/v1/zonas/` | CRUD |
| Farmacias | `/api/v1/farmacias/` | CRUD; distancia mayor que cero |
| Ventas | `/api/v1/ventas/` | Lectura; cliente, detalles y productos anidados |
| Evaluaciones | `/api/v1/evaluaciones/` | Lectura; zona anidada |

Los detalles usan `<id>/`. Las vistas genéricas del ejercicio 4 se conservan bajo `/api/v1/basico/productos/`, sin
duplicar las rutas del router principal. Se mantiene `/api/v1/` del repositorio actual en lugar del `/api/` orientativo
del PDF.

### Filtros y acciones

- Productos: `?search=Analgesico`, `?ordering=precio`, `?disponible=true` y acción `con-stock/`.
- Farmacias: `?search=Botica`, `?ordering=distancia_metros`, `?estado=Activa` y acción `activas/`.
- Los parámetros pueden combinarse con `&`. Las acciones reutilizan `ProductoQuerySet.con_stock()` y
`FarmaciaQuerySet.activas()` y conservan filtros/paginación.
- Ventas: `select_related('cliente')` y `prefetch_related('detalles__producto__categoria')`.
- Evaluaciones: `select_related('zona')`.
```

E14_C03_git_status Cambios listos para revisión.

Cambios revisados antes del commit

FarmaPoint · Lab 08 · E14_C03_git_status
Salida real de git status --short

```
$ git status --short
 M README.md
 M farmacia_project/LAB08_EJERCICIO01.md
 M farmacia_project/LAB08_EVIDENCIAS.md
 M farmacia_project/README.md
 M farmacia_project/src/config/settings.py
 M farmacia_project/src/config/urls.py
 M farmacia_project/src/farmacia/api_urls.py
 M farmacia_project/src/farmacia/api_views.py
 M farmacia_project/src/farmacia/models.py
 M farmacia_project/src/farmacia/serializers.py
?? farmacia_project/src/farmacia/management/__init__.py
?? farmacia_project/src/farmacia/management/commands/__init__.py
?? farmacia_project/src/farmacia/management/commands/sembrar_lab08.py
?? farmacia_project/src/farmacia/test_lab08_completo.py
```

E14_C04_commit_push Evidencia de commit y push después de realizarlos.

Publicación del código en GitHub

FarmaPoint · Lab 08 · E14_C04_commit_push
Salida real de commit y push autorizados

```
$ git commit -m Completa Lab 08 con API de ubicaciones validacion y pruebas
[main c5144bd] Completa Lab 08 con API de ubicaciones validacion y pruebas
 13 files changed, 296 insertions(+), 293 deletions(-)
 create mode 100644 farmacia_project/src/farmacia/management/__init__.py
 create mode 100644 farmacia_project/src/farmacia/management/commands/__init__.py
 create mode 100644 farmacia_project/src/farmacia/management/commands/sembrar_lab08.py
 create mode 100644 farmacia_project/src/farmacia/test_lab08_completo.py

$ git push origin main
To https://github.com/SAguirreT/django_project.git
   8c901e4..c5144bd  main -> main
```

E14_pruebas E14_pruebas

Pruebas de publicación y copia local

FarmaPoint · Lab 08 · E14_pruebas
Salidas reales de pruebas independientes

```
PUBLICACIÓN
ducto_crud_http_y_validacion (farmacia.tests_api.FarmaciaApiTests.test_producto_crud_http_y_validacion) ... ok
test_venta_anidada_solo_lectura_y_consultas_optimizadas
(farmacia.tests_api.FarmaciaApiTests.test_venta_anidada_solo_lectura_y_consultas_optimizadas) ... ok
test_vistas_genericas_list_create_y_detalle
(farmacia.tests_api.FarmaciaApiTests.test_vistas_genericas_list_create_y_detalle) ... ok

----------------------------------------------------------------------
Ran 25 tests in 3.898s

OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...


COPIA LOCAL
ducto_crud_http_y_validacion (farmacia.tests_api.FarmaciaApiTests.test_producto_crud_http_y_validacion) ... ok
test_venta_anidada_solo_lectura_y_consultas_optimizadas
(farmacia.tests_api.FarmaciaApiTests.test_venta_anidada_solo_lectura_y_consultas_optimizadas) ... ok
test_vistas_genericas_list_create_y_detalle
(farmacia.tests_api.FarmaciaApiTests.test_vistas_genericas_list_create_y_detalle) ... ok

----------------------------------------------------------------------
Ran 32 tests in 4.025s

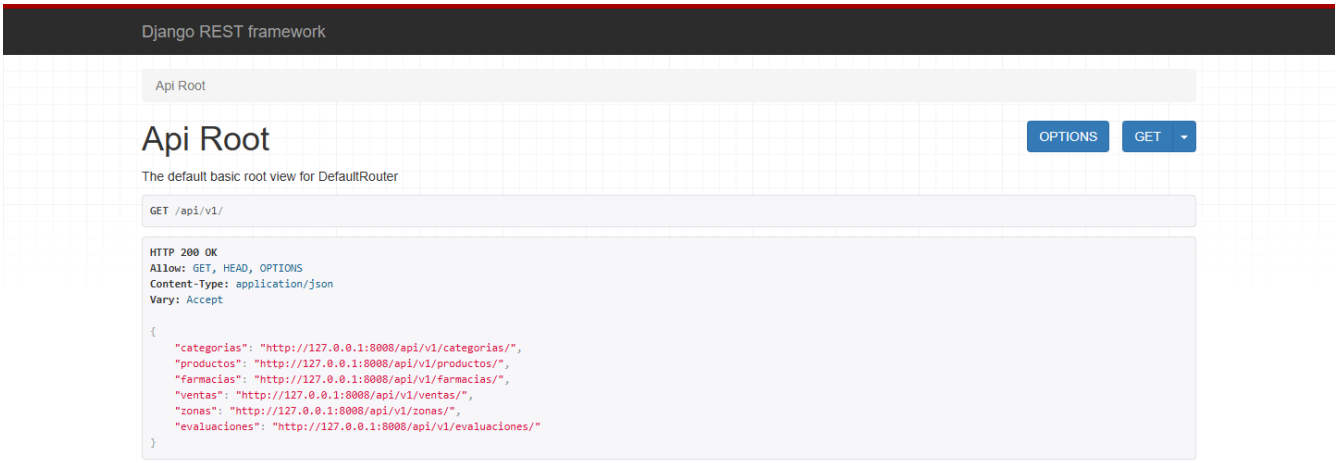
OK
Destroying test database for alias 'default' ('file:memorydb_default?mode=memory&cache=shared')...


ESQUEMA
No changes detected
```

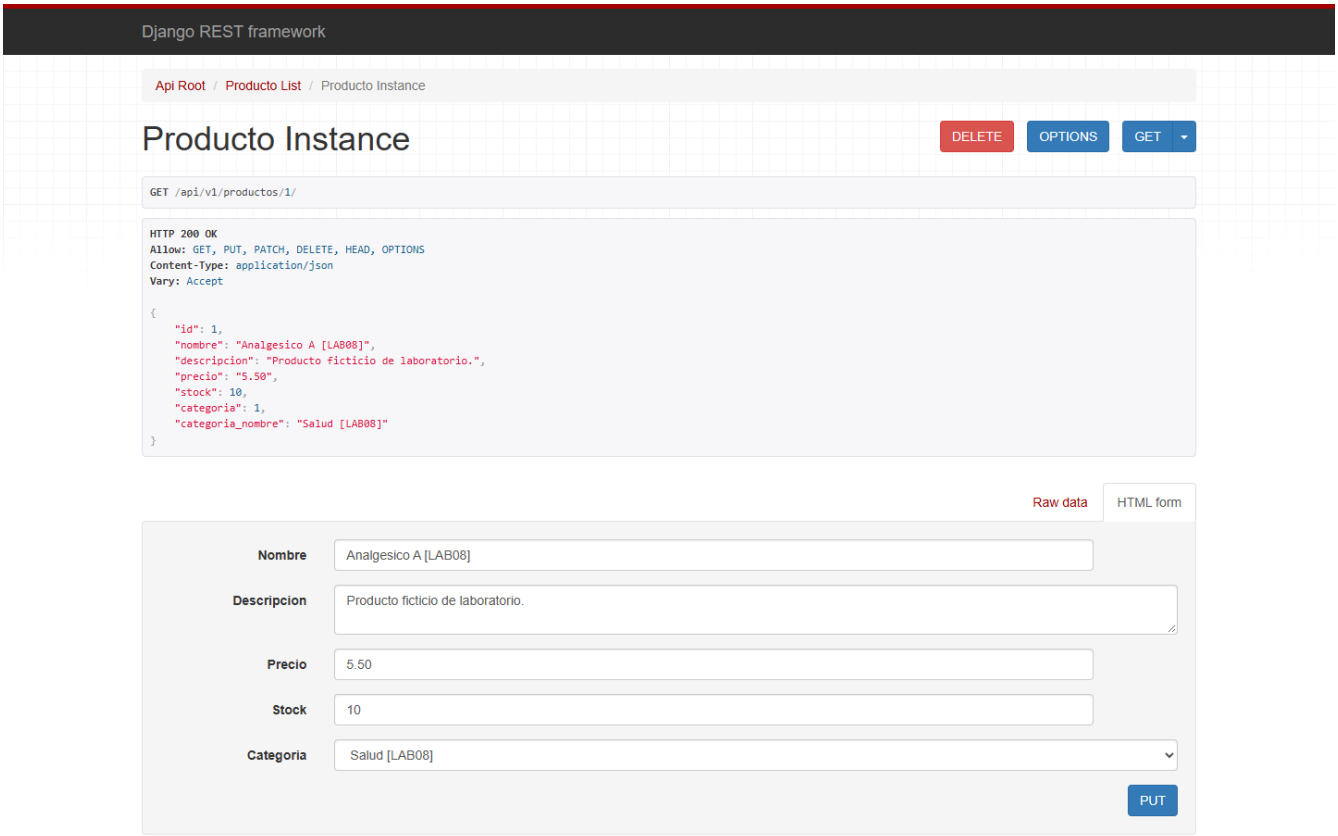
Comprobación visual del navegador

Las siguientes capturas son páginas reales de DRF y de la interfaz HTML consultadas en http://127.0.0.1:8008. La venta de muestra muestra dos productos y un total de 17.50.

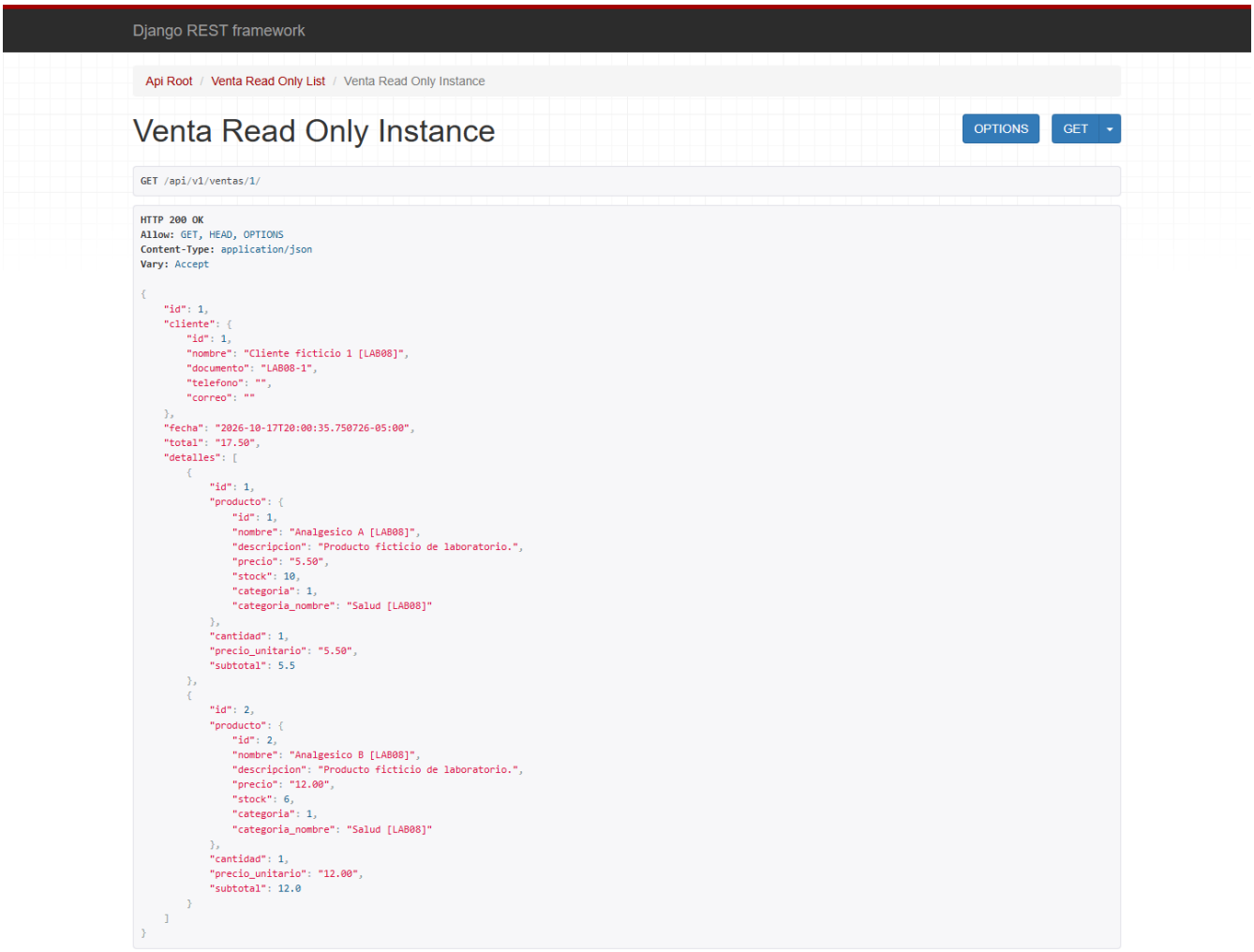
API_navegador Índice de la API con seis recursos.



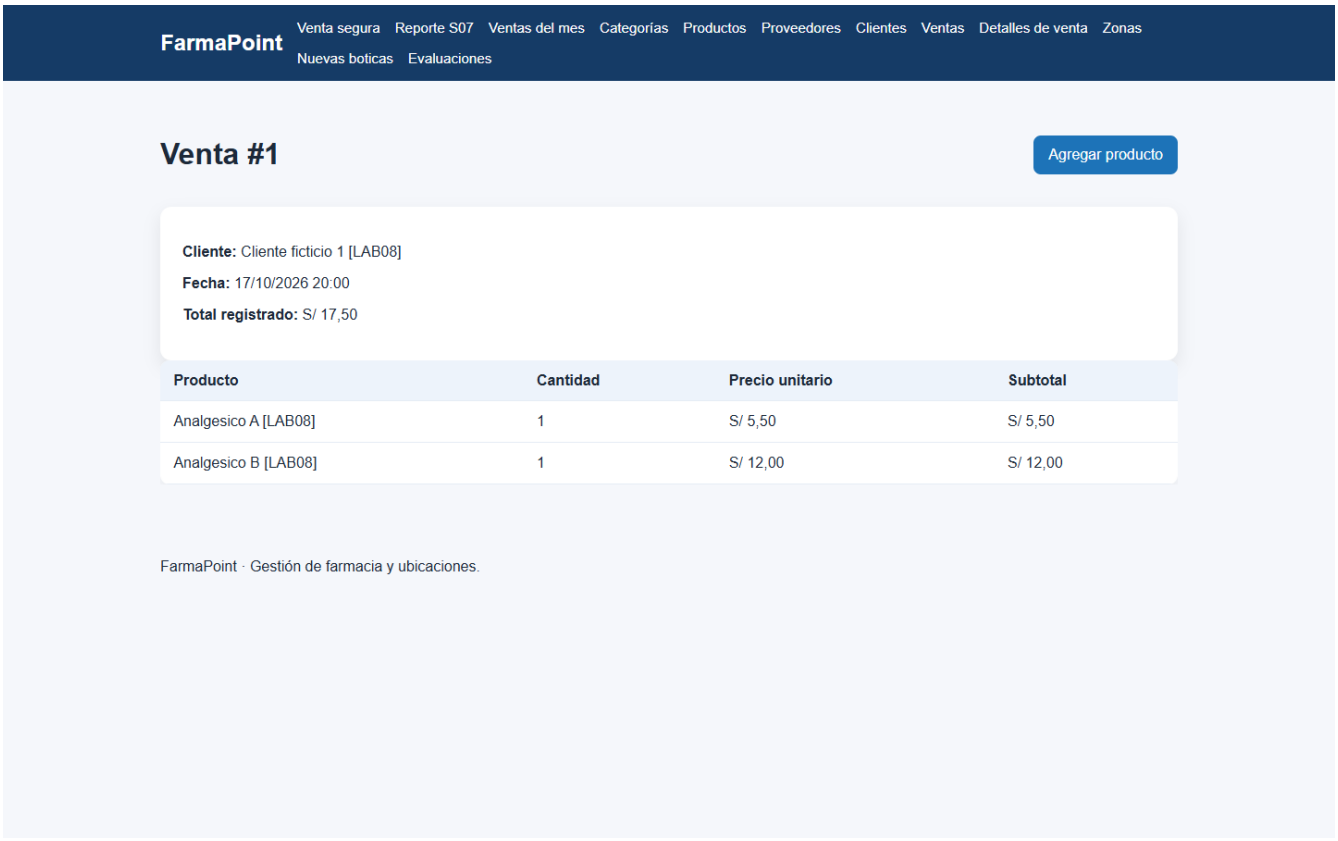
Productos_navegador Detalle de producto en la interfaz navegable DRF.



Venta_navegador Respuesta de solo lectura con cliente y productos anidados.



HTML_navegador La vista HTML de venta continúa disponible.



Conclusiones

FarmaPoint expone seis recursos mediante una API versionada. Productos, categorías, zonas y farmacias admiten CRUD; ventas y evaluaciones son de solo lectura y muestran relaciones anidadas. Los serializers rechazan precios y distancias no positivos antes de guardar.

Las comprobaciones HTTP produjeron los códigos 200, 201, 400, 404, 204 y 405 en los escenarios documentados. La igualdad de los JSON antes y después de precargar relaciones confirma que la optimización reduce consultas sin cambiar la información presentada.

Se conservaron el HTML y Django Admin. Los QuerySets personalizados se reutilizan mediante acciones y filtros combinables. El código del laboratorio está publicado y cuenta con pruebas automatizadas; la configuración continúa siendo de desarrollo local.

Cómo reproducir

1. Clonar el repositorio y crear un entorno virtual. Instalar farmacia_project/src/requirements.txt.

2. Desde farmacia_project/src, ejecutar migrate y sembrar_lab08. Para aislar los datos, definir FARMAPOINT_DB con la ruta de una SQLite de demostración.

3. Ejecutar manage.py check y manage.py test farmacia. Iniciar runserver 127.0.0.1:8008.

4. Abrir /api/v1/ e importar lab08/FarmaPoint_Lab08.postman_collection.json. Ajustar categoria_id y zona_id si se usa otra base.

5. lab08/recolectar.py reproduce las peticiones y mediciones sobre lab08/demo.sqlite3; requiere que esa base haya sido migrada y sembrada. Los datos y salidas de referencia están en evidencias/.

Referencias

Repositorio y código publicado:
https://github.com/SAguirreT/django_project/commit/c5144bd377d45efcff114b49ab1e0249ed589071

Documento de trabajo: GLAB_S08_django.pdf, proporcionado por el usuario

Django REST Framework: https://www.django-rest-framework.org/

ViewSets: https://www.django-rest-framework.org/api-guide/viewsets/

Filtros: https://www.django-rest-framework.org/api-guide/filtering/